Connecting Our Rails App to Postgres

We've just seen that our database is up and running and reachable from other containers in our app's network. However, before we can start making use of it, we have to configure our Rails app to connect to it.

Let's do this now.

Installing the Postgres Gem

First things first. In order to get our Rails app talking to Postgres, we need to install the Postgres client gem. Open up your Gemfile and update it to replace:

```
gem 'sqlite3'
```

with:

```
gem 'pg', '~> 1.0'
```

To actually install the new gem, we need to run bundle install, which we do by rebuilding our image (we discuss gem management further on page 113). Let's first stop our Rails server:

```
$ docker-compose stop web
```

and then rebuild our image:

```
$ docker-compose build web
Building web
Step 1/8 : FROM ruby:2.6
«...»
Step 6/8 : RUN bundle install
«...»
Installing pg 1.1.4 with native extensions
«...»
Bundle complete! 16 Gemfile dependencies, 69 gems now installed.
Bundled gems are installed into `/usr/local/bundle`
«...»
Removing intermediate container 9b01b1fa29fc
 ---> f9e6330d40b6
Step 7/8 : COPY . /usr/src/app/
 ---> 70fb0e2e0091
Step 8/8 : CMD ["bin/rails", "s", "-b", "0.0.0.0"]
 ---> Running in 16cc0923b855
Removing intermediate container 16cc0923b855
 ---> d4ffbe8f72d3
Successfully built d4ffbe8f72d3
Successfully tagged myapp_web:latest
```

With the Postgres gem installed, we can move on to configuring our database.yml.